

Project Report: Ride-Sharing Analytics & Driver Performance Pipeline

Devansh Saini

Data Engineering Enthusiast | B.Tech Computer Science Engineering
Technologies: Databricks, PySpark, Delta Lake, Python

1 Problem Statement

Ride-sharing platforms generate large volumes of trip and driver data. However, raw data on its own does not provide actionable insights. There is a critical need for a structured data engineering pipeline to analyze driver performance, cancellations, trip delays, and passenger demand patterns. This project solves that by building a robust pipeline that processes raw ride data and generates meaningful, business-ready insights.

2 Project Objectives

- **Track Activity:** Monitor ride and driver activity across the platform.
- **Analyze Inefficiencies:** Analyze ride cancellations and trip delays.
- **Fleet Optimization:** Identify high-demand pickup locations to optimize fleet distribution.
- **Performance Evaluation:** Evaluate driver performance and rank them by revenue generation.
- **Data Validation:** Ensure data consistency across all processing layers.
- **Optimization:** Apply Spark optimization techniques (e.g., Broadcast Joins) for scalable processing.

3 Data Architecture (Medallion Architecture)

The project strictly follows the **Medallion Architecture**, implemented via a unified Databricks single-notebook workflow.

- **Bronze Layer:** Ingests and stores raw CSV datasets (drivers, trips, trip logs) exactly as they are provided, maintaining a strict historical source of truth.
- **Silver Layer:** Transforms the raw data into a clean, structured, and reliable dataset. This layer performs data cleansing, handles missing values, filters invalid records, and executes optimized joins.
- **Gold Layer:** Generates aggregated, business-ready KPIs and analytics tables tailored for reporting, dashboarding, and stakeholder consumption.

4 Dataset Schemas

4.1 Drivers Dataset (drivers.csv)

Column	Data Type	Description
driver_id	INT	Unique driver ID
name	STRING	Driver name
city	STRING	Operating city
rating	FLOAT	Driver rating

4.2 Trips Dataset (trips.csv)

Column	Data Type	Description
trip_id	INT	Trip ID
driver_id	INT	Driver ID
pickup_location	STRING	Pickup location
drop_location	STRING	Drop location
distance_km	FLOAT	Distance in kilometers
fare_amount	FLOAT	Fare amount
trip_status	STRING	Trip status (Completed, Cancelled)

4.3 Trip Logs Dataset (trip_logs.csv)

Column	Data Type	Description
log_id	INT	Unique log ID
trip_id	INT	Trip ID reference
start_time	TIMESTAMP	Trip start time
end_time	TIMESTAMP	Trip end time
delay_minutes	FLOAT	Delay in minutes
cancellation_flag	INT	1 if cancelled, 0 otherwise

5 Pipeline Implementation & Execution

The entire pipeline was built and executed in Databricks using PySpark. Below is the step-by-step implementation mapping to the project tasks.

5.1 Bronze Layer: Raw Data Ingestion

The pipeline begins by reading the raw datasets natively from the Databricks Catalog and saving them as Delta tables without any transformations.

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.window import Window
```

```
spark = SparkSession.builder.appName("RideSharing_Medallion_Pipeline").getOrCreate()
```

```
# Task 1: Read all datasets
```

```

drivers_df = spark.table("any.default.drivers")
trip_logs_df = spark.table("any.default.trip_logs")
trips_df = spark.table("any.default.trips")

# Task 2: Store raw data into Bronze layer
drivers_df.write.format("delta").mode("overwrite").saveAsTable(
    "any.default.bronze_drivers")
trip_logs_df.write.format("delta").mode("overwrite").saveAsTable(
    "any.default.bronze_trip_logs")
trips_df.write.format("delta").mode("overwrite").saveAsTable(
    "any.default.bronze_trips")

```

5.2 Silver Layer: Cleansing, Transformation & Joins

Data is read from the Bronze layer and enriched. A **Broadcast Join** is utilized to optimize the joining of the smaller drivers dimension table with the larger trips fact table.

```

# Read from Bronze layer
bronze_drivers = spark.table("any.default.bronze_drivers")
bronze_trip_logs = spark.table("any.default.bronze_trip_logs")
bronze_trips = spark.table("any.default.bronze_trips")

# Task 3 & 14: Perform joins using Broadcast Join optimization
joined_df = bronze_trips \
    .join(F.broadcast(bronze_drivers), "driver_id", "left") \
    .join(bronze_trip_logs, "trip_id", "left")

# Task 4 & 6: Clean data, handle nulls, and filter invalid data
cleaned_df = joined_df \
    .filter(F.col("distance_km") >= 0) \
    .filter((F.col("trip_status") == "Cancelled") |
            (F.col("start_time").isNotNull() & F.col("end_time").isNotNull())) \
    .fillna({"delay_minutes": 0, "fare_amount": 0.0})

# Task 5: Create derived columns
silver_df = cleaned_df \
    .withColumn("is_completed", F.when(F.col("trip_status") == "Completed", 1)
    .otherwise(0)) \
    .withColumn("trip_duration_mins", \
        F.round((F.unix_timestamp("end_time") -
        F.unix_timestamp("start_time")) / 60, 2))

# Task 7: Write cleaned data into Silver layer
silver_df.write.format("delta").mode("overwrite").saveAsTable(
    "any.default.silver_enriched_trips")

```

Note: Schemas were generated and verified successfully in Databricks.

5.3 Gold Layer: Aggregated KPIs and Analytics

Using the trusted Silver data, the pipeline creates highly aggregated views. Window functions rank drivers by their total revenue generation within their respective cities.

```

# Read from Silver Layer
silver_data = spark.table("any.default.silver_enriched_trips")

# Task 8, 9, 11: Driver performance, cancellation rates, and revenue
driver_metrics_df = silver_data.groupBy("driver_id", "name", "city", "rating").agg(
    F.count("trip_id").alias("total_trips"),
    F.sum("is_completed").alias("completed_trips"),
    F.sum("cancellation_flag").alias("cancelled_trips"),
    F.round((F.sum("cancellation_flag") / F.count("trip_id")) * 100, 2)
        .alias("cancellation_rate_pct"),
    F.round(F.sum("fare_amount"), 2).alias("total_revenue"),
    F.round(F.avg("delay_minutes"), 2).alias("avg_delay_mins")
)

# Task 15: Use window functions to rank drivers based on performance
window_spec = Window.partitionBy("city").orderBy(F.col("total_revenue").desc())
driver_performance_gold = driver_metrics_df \
    .withColumn("city_rank_by_revenue", F.rank().over(window_spec))

# Task 10: Identify high-demand pickup locations
high_demand_locations_gold = silver_data.groupBy("pickup_location").agg(
    F.count("trip_id").alias("total_requests"),
    F.sum("is_completed").alias("successful_pickups"),
    F.round(F.sum("fare_amount"), 2).alias("revenue_generated")
).orderBy(F.col("total_requests").desc())

# Task 12: Store aggregated results in Gold layer
driver_performance_gold.write.format("delta").mode("overwrite").saveAsTable(
    "any.default.gold_driver_performance")
high_demand_locations_gold.write.format("delta").mode("overwrite").saveAsTable(
    "any.default.gold_high_demand_locations")

```

5.4 Data Quality Validation

To ensure data consistency and that no duplicates were accidentally created during our table joins, an automated validation check is performed.

```

# Task 13: Validate data consistency across layers
bronze_count = spark.table("any.default.bronze_trips").count()
silver_count = spark.table("any.default.silver_enriched_trips").count()

print(f"Total Raw Trips (Bronze): {bronze_count}")
print(f"Total Valid Trips (Silver): {silver_count}")

if silver_count <= bronze_count:
    print("Data validation passed successfully.")
else:
    print("WARNING: Silver layer has more records than Bronze.")

```

Validation Output:

- Total Raw Trips (Bronze): 150

- Total Valid Trips (Silver): 150
- Data validation passed successfully. Silver records do not exceed Bronze records.

6 Future Improvements

To further scale and productionize this pipeline, the following enhancements are recommended:

1. **Pipeline Orchestration:** Transition the single notebook to a scheduled Databricks Workflow or orchestrate via Apache Airflow.
2. **Streaming Ingestion:** Implement Databricks Auto Loader for real-time processing of `trip_logs` instead of batch processing.
3. **BI Integration:** Connect the Gold tables directly to Power BI or Tableau to create live, interactive dashboards for stakeholders.
4. **Data Governance:** Integrate Unity Catalog to manage column-level access controls and data lineage mapping.